The Lean Programming Language and Theorem Prover

Leo de Moura
Senior Principal Applied Scientist, AWS
Chief Architect, Lean FRO
ETAPS | April 13, 2026

Last Month, Something Unexpected Happened

A **general-purpose AI**, with no special training for theorem proving, **converted zlib (a C compression library) to Lean**, passed the test suite, and then **proved** that the code is **correct**.

Not tested. Proved. For every possible input. **lean-zip**

"This was not expected to be possible yet."

How? Let me show you.

What is Lean?

- A **programming language** and a **proof assistant**
- Same language for code and proofs. No translation layer.
- Small trusted kernel. Proofs can be exported and independently checked.
- Lean is implemented in Lean. **Very extensible**.
- The math community made Lean their own: >50,000 lines of extensions.

```
def reverse : List α → List α
  | [] => []
  | x :: xs => reverse xs ++ [x]

theorem reverse_append (xs ys : List α)
    : reverse (xs ++ ys) = reverse ys ++ reverse xs := by
  induction xs with
  | nil => simp [reverse]
  | cons x xs ih => simp [reverse, ih]

#eval reverse [1, 2, 3]
```

```
[3, 2, 1]
```

Propositions as Types

```
def odd (n : Nat) : Prop := ∃ k, n = 2 * k + 1
```

- The type of `odd 3` is `Prop`, a statement that can be true or false.
- We can't evaluate it: `odd` returns a `Prop`, not a `Bool`.
- But we can **prove** it by providing a witness.

```
example : odd 3 := ⟨1, by decide⟩
```

Theorem Proving Is a Game

> *"You have written my favorite computer game" — Kevin Buzzard*

The "game board": you see goals and hypotheses, then apply "moves" (tactics).

```
theorem square_of_odd_is_odd : odd n → odd (n * n) := by
  intro ⟨k₁, e₁⟩
  simp [e₁, odd]
  exists 2 * k₁ * k₁ + 2 * k₁
  grind
```

Each tactic transforms the goal until nothing remains. The kernel checks the final proof term.

The Kernel Catches Mistakes

```
example : odd 3 := ⟨2, by decide⟩
```

You don't need to trust me, or my automation. You only need to trust the small kernel.

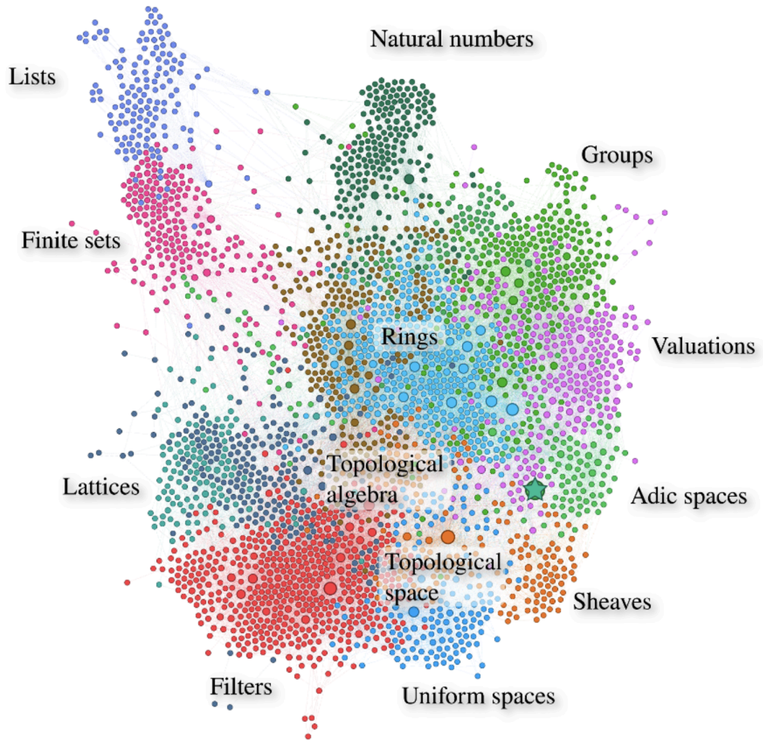
An incorrect proof is rejected immediately.

Lean has **multiple independent kernels**.

You can build your own and submit it to **arena.lean-lang.org**.

Success Stories: Mathlib

- 200,000+ theorems. 750+ contributors.
- From basic algebra to modern research mathematics.
- The dataset every AI math system trains on.



Success Stories: Mathematics

Six Fields Medalists engaged: Tao, Scholze, Viazovska, Gowers, Hairer, Freedman.

- Tao: **Equational Theories Project**, **PFR project**
- Scholze: **Liquid Tensor Experiment**
- Buzzard: **Fermat's Last Theorem**
- Mochizuki: Formalization of IUT — views Lean as a tool for communication
- **Carleson's Theorem** (completed)
- **Fields Medal sphere packing** (AI assisted)

The Liquid Tensor Experiment

In 2020, Peter Scholze posed a formalization challenge.

> *"I spent much of 2019 obsessed with the proof of this theorem, almost getting crazy over it. I still have some small lingering doubts." — Peter Scholze*

Johan Commelin led a team that verified the proof, with only minor corrections.

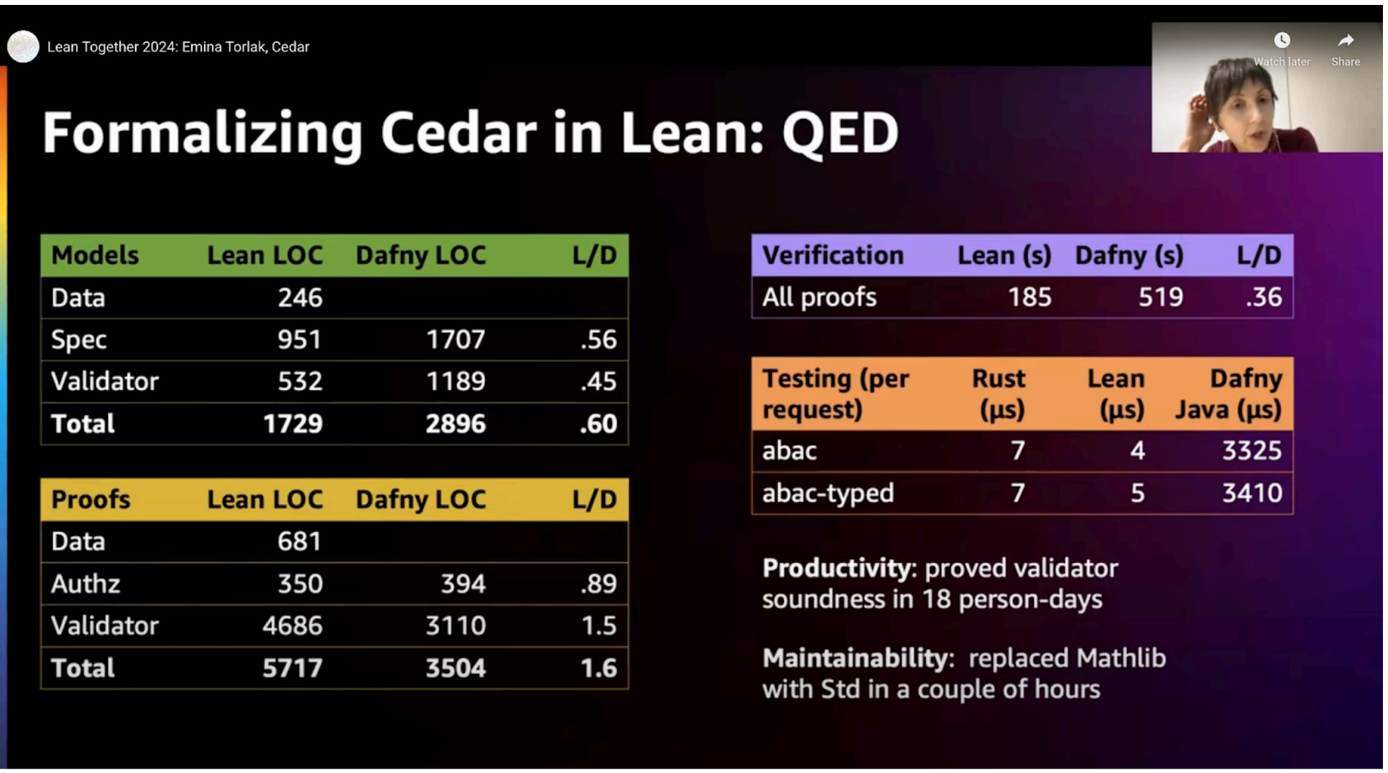
They verified and simplified it without fully understanding the proof. Lean was a guide.

> *"The Lean Proof Assistant was really that: an assistant in navigating through the thick jungle that this proof is." — Peter Scholze*

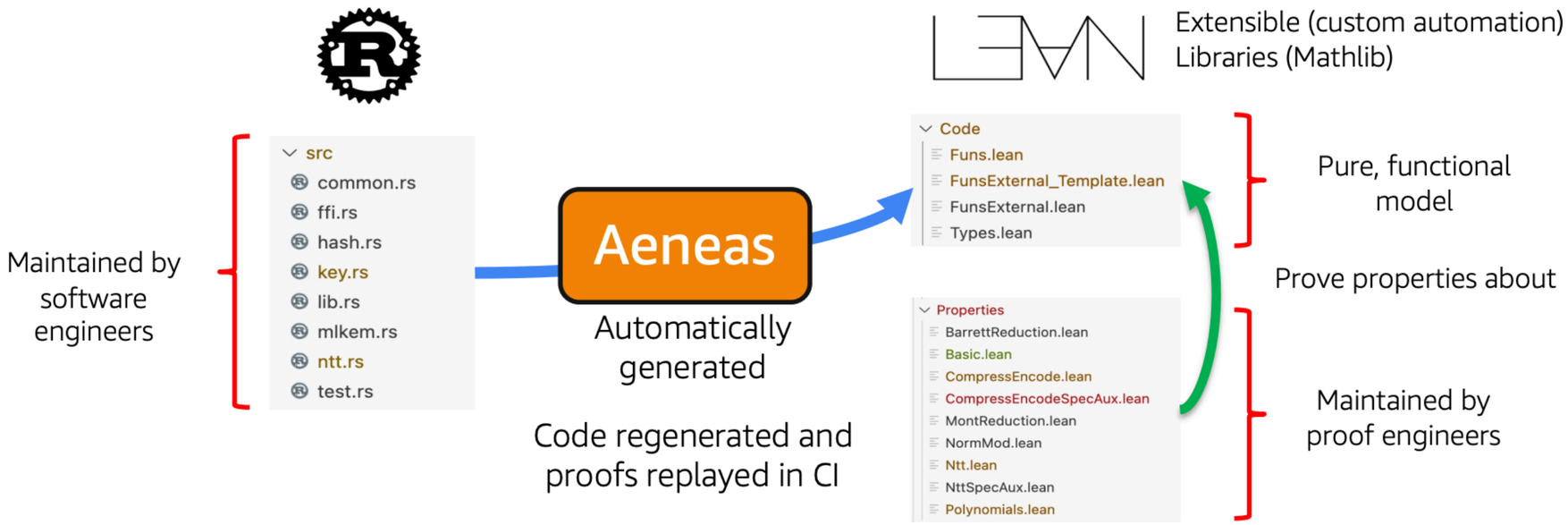
Cedar (AWS)

- **Authorization policy engine.** Verified in Lean.
- No version ships unless proofs pass.

> *"We've found Lean to be a great tool for verified software development." — Emina Torlak*



SymCrypt (Microsoft)



- **Core cryptographic library**. Verified using Aeneas + Lean.
- They verify the Rust code as written by software engineers.

Veil: Verified Distributed Protocols

- **Built on Lean by Ilya Sergey's group**.
- Combines model checking with full formal proof.
- Found an inconsistency in a prior formal verification of Rabia consensus that went undetected across two separate tools.

```
action advanceRound {
  require round < t + 1

  let deadToAliveDelivery : node → node → Bool :| true

  seen N V := seen N V ||
    alive N &&
      decide ((∃ m, alive m ∧ seen m V) ∨
              (∃ d, crashedInThisRound d ∧ deadToAliveDelivery d N ∧ seen d V))

  crashedInThisRound N := false
  round := round + 1
}
```

Success Stories: AI

To date, every AI system that solved IMO problems with formal proofs used Lean.

- **AlphaProof** (Google DeepMind) — silver medal, IMO 2024
- **Aristotle** (Harmonic) — gold medal, IMO 2025
- **SEED Prover** (ByteDance) — silver medal, IMO 2025

Three independent teams. No competing platform was used by any of them.

AI Scoreboard

- **Axiom:** 12/12 on Putnam 2025.
- **DeepSeek Prover-V2:** 88.9% on MiniF2F-test. Open-source, 671B parameters.
- **Harmonic:** Built the Aristotle AI — gold medal, IMO 2025

None of these systems existed in early 2024.

Harmonic

Axiom

Math, Inc.

Logical Intelligence

The Kernel as AI Safety Infrastructure

> *"It's really important with these formal proof assistants that there are no backdoors or exploits you can use to somehow get your certified proof without actually proving it, because reinforcement learning is just so good at finding these backdoors." — Terence Tao*

The kernel's soundness is not just a theoretical property. It is **AI safety infrastructure**.

RL agents will find every shortcut. The kernel is the one thing they can't fake.

Who Watches the Provers? | Comparator | Validating a Lean Proof

Radix: 10 AI Agents, One Weekend

10 AI agents built a **verified embedded DSL** with proved optimizations in **a single weekend**.

- Zero `sorry` — 52 theorems, all complete
- 5 verified compiler optimizations
- Determinism, type safety, memory safety — all proved
- Interpreter correctness — sound and complete
- 27 modules, ~7,400 lines of Lean
- I did not touch the code. Zero lines. Full agent autonomy.

github.com/leodemoura/RadixExperiment

Today we'll build a simplified version of Radix together.

Do We Still Need Proof Automation?

> *"I thought AI would prove all theorems for us now."*

- When `grind` closes a goal in one step instead of fifty, the AI's search tree shrinks.
- Compact proofs = better training data = better AI provers.
- `grind` is a new tactic inspired by SMT solvers: congruence closure, E-matching, linear arithmetic — all cooperating.

```
-  refine (_, _, _, _, ha, haj, hb, hbj, hc, hcj, hd, hdj, ?_, ?_, ?_, ?_, ?_)
-    <;> rw [mem_insert] at * <;> try rintro rfl
-  · obtain (rfl | ha) := ha
-    · obtain (rfl | hb) := hb
-      · exact hw.isPathGraph3Compl.fst_ne_snd rfl
-      · exact hw.fst_notMem_right hb
-    · obtain (rfl | hb) := hb
-      · exact hw.snd_notMem_left ha
-      · exact haj <| hW <| mem_inter_of_mem ha hb
-  · obtain (rfl | ha) := ha
-    · obtain (rfl | hd) := hd
-      · exact hw.isPathGraph3Compl.ne_fst rfl
-      · exact hw.fst_notMem_right hd
-    · obtain (rfl | hd) := hd
-      · exact hw.notMem_left ha
-      · exact haj <| hW <| mem_inter_of_mem ha hd
-  · obtain (rfl | hb) := hb
-    · obtain (rfl | hc) := hc
-      · exact hw.isPathGraph3Compl.ne_snd rfl
-      · exact hw.snd_notMem_left hc
-    · obtain (rfl | hc) := hc
-      · exact hw.notMem_right hb
-      · exact hbj <| hW <| mem_inter_of_mem hc hb
-  · intro hat
-    obtain (rfl | ha) := ha
-    · exact hw.fst_notMem_right hat
-    · exact haj <| hW <| mem_inter_of_mem ha hat
-  · intro hbs
-    obtain (rfl | hb) := hb
-    · exact hw.snd_notMem_left hbs
-    · exact hbj <| hW <| mem_inter_of_mem hbs hb
+  exact (_, _, _, _, ha, haj, hb, hbj, hc, hcj, hd, hdj, by grind)
```

Comment on line R312

YaelDillies 19 minutes ago — Collaborator

Wow! 😍

What is grind?

New proof automation (since Lean v4.22), developed by Kim Morrison and me.

A proof-automation tactic **inspired by modern SMT solvers**. Think of it as a **virtual whiteboard**:

- Discovers new equalities, inequalities, etc.
- Writes facts on the board and merges equivalent terms
- Multiple engines cooperate on the same workspace

Cooperating Engines:

- Congruence closure; E-matching; Constraint propagation; Guided case analysis
- Satellite theory solvers (linear integer arithmetic, commutative rings, linear arithmetic, AC)

Supports dependent types, type-class system, and dependent pattern matching

Produces ordinary Lean proof terms for every fact.

Tao on grind

Terence Tao
@tao@mathstodon.xyz

In contrast, AI chatbots are usually tuned to avoid a "failure mode" as much as possible, at the expense of increasing the occurrence of "intermediate modes" where the chatbot response looks potentially useful, and invites further interaction from the user, but is not exactly providing what the user wants, and could contain hallucinations or some fundamental misunderstanding of the task that would take significant effort to uncover. Paradoxically, such tools may become significantly more useful if they simply reported that they were unable to provide a high quality answer to a query in such cases.

A comparison may be drawn with the increasingly advanced, but stringently verified, "tactics" used in a modern proof assistant such as Lean. I have been experimenting recently with the new tactic `grind` in Lean, which is a powerful tool (inspired more by "good old-fashioned AI" such as satisfiability modulo theories (SMT) solvers, than modern data-driven AI) to try to close complex proof goals if all the tools needed to do so are already provided in the proof environment; roughly speaking, this corresponds to proofs that can be obtained by "expanding everything out and trying all obvious combinations of the hypotheses". When I apply `grind` to a given subgoal, it can report a success within seconds, closing that subgoal in a Lean-verified fashion and allowing me to move on to the next subgoal. But, importantly, when this does not work, I quickly get a "`grind` failed" message, in which case I simply delete `grind` from the code and proceed by a more pedestrian sequence of lower level tactics. (2/3)

> *"I have been experimenting recently with the new tactic* `grind` *in Lean, which is a powerful tool inspired by 'good old-fashioned AI' such as satisfiability modulo theories (SMT) solvers... When I apply* `grind` *to a given subgoal, it can report a success within seconds... But, importantly, when this does not work, I quickly get a* `grind` *failed message."*

How Does AI Affect Proof Automation?

- I built heuristics for Z3: quantifier instantiation, case-split ordering, proof search.
- AI can do this better. AI replaces the heuristics.
- The algorithmic core (congruence closure, linear arithmetic, decision procedures) is more valuable than ever.
- `grind` is designed this way: strong algorithms, AI-replaceable search.

```
example : (cos x + sin x)^2 = 2 * cos x * sin x + 1 := by
  grind =>
    use [trig_identity]
    ring
```

The Ecosystem

elan: The Lean Toolchain Manager

- Manages Lean versions (like `rustup` for Rust or `ghcup` for Haskell)
- `lean-toolchain` file pins the version per project
- Installation: `curl https://elan.lean-lang.org/install.sh -sSf | sh`
- Each project can use a different Lean version. `elan` switches automatically.

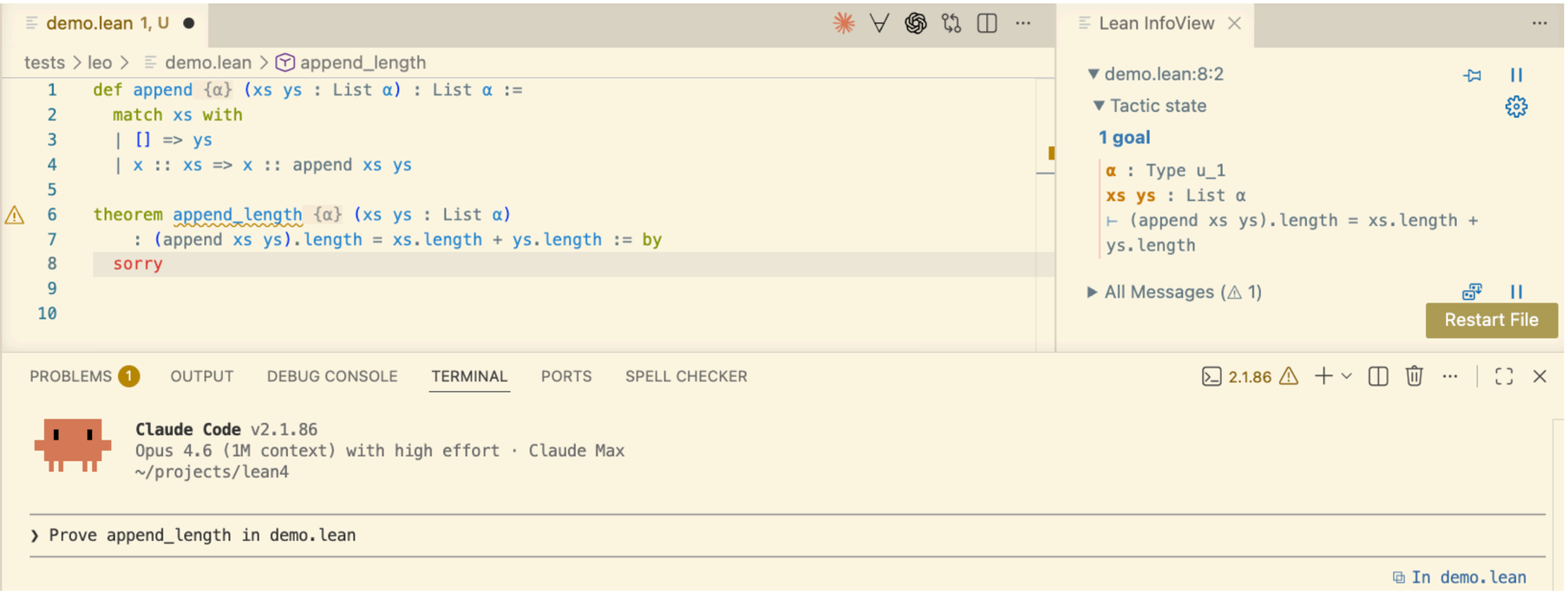
Lake: The Lean Build System

- Lakefile is written in Lean. Build configuration is a Lean program. **Manual**
- Dependency management from Git repos.
- `lake new` creates an initial Lean package in a new directory.
- `lake build` compiles everything.
- `lake exe <name>` looks for the executable target exe-target in the workspace, builds it if it is out of date, and then runs it with the given args in Lake's environment.

```
import Lake
open System Lake DSL

require «verso-slides» from git
  "https://github.com/leanprover/verso-slides.git"@"main"

package «etaps-tutorial» where
  version := v!"0.1.0"

lean_lib MiniRadix
lean_lib Slides

@[default_target] lean_exe «etaps-tutorial» where root := `Main
```

VS Code Extension

- Real-time feedback: errors, goals, and hints **as you type**.
- Goal state panel: see what you need to prove at each tactic step.
- Hover information: types, docstrings, source locations.
- Go to definition. Find references. Code actions.
- 200,000+ installs.



The IDE is where you collaborate with AI now.

VS Code Extension

I asked Claude to prove `append_length`. It wrote the proof, Lean checked it. No goals remaining.

Lean Zulip: The Community Hub

- **leanprover.zulipchat.com**
- Active community of mathematicians, CS researchers, and engineers.
- Streams: `#general`, `#new members`, `#mathlib`, `#lean4`
- Best place to ask questions and get help.
- Searchable archive — many common questions already answered.

Learning Resources

- **Functional Programming in Lean** — David Thrane Christiansen
- **Mathematics in Lean** — Jeremy Avigad, Patrick Massot
- **Theorem Proving in Lean 4** — Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich
- **Lean Reference Manual** — comprehensive, updated with each release
- **Metaprogramming in Lean 4** — community effort

Loogle: Search for Lean Theorems

- **loogle.lean-lang.org**
- Search by type signature: `List.map (_ ++ _)` finds relevant lemmas
- Indispensable for finding theorems in Mathlib's 200,000+ entries
- Also available as a Zulip bot: just type `#loogle` followed by your query

Verso: Written in Lean. Checked by Lean.

- Future math papers will have Lean code that is checked.
- Lecture notes, papers, and slides — all in one tool.
- **These slides** are written in **Verso**.
- **lean-lang.org** is written in Verso.
- Pure Lean. Built by David Thrane Christiansen (Lean FRO).

````
The type of all predicates over a type is that
type's _powerset_. Elements of one of these
subsets satisfy the predicate:
```lean
def Set (α : Type u) : Type u :=
  α → Prop

instance : Membership α (Set α) where
  mem xs x := xs x
```
A function $`f` is surjective if each element
of the range is covered by an element of the
domain:
```lean
def Surjective (f : α → β) :=
  ∀ y, ∃ x, f x = y
```
:::theorem "Cantor"
Given a function $`f ∈ S → \mathcal{P}(S)`,
[Cantor's diagonal argument]
(https://en.wikipedia.org/wiki/Cantor%27s_diago
nal_argument) involves the diagonal set $`\{x ∈
S \mid x \not\in f(x)\}`. The {tactic}`grind`
tactic takes care of many reasoning steps:
```lean
theorem cantor (f : S → Set S) : ¬ Surjective f :=
  intro h
  have ⟨x, p⟩ := h (fun x : S => x ∉ f x)
  have : x ∈ f x ↔ x ∉ f x := by
    constructor <;>
    simp [Membership.mem] at * <;>
    grind
  grind
```
:::
````

The type of all predicates over a type is that type's *powerset*. Elements of one of these subsets satisfy the predicate:

```
def Set (α : Type u) : Type u :=
  α → Prop

instance : Membership α (Set α) where
  mem xs x := xs x
```

A function f is surjective if each element of the range is covered by an element of the domain:

```
def Surjective (f : α → β) :=
  ∀ y, ∃ x, f x = y
```

Theorem: *Cantor*

Given a function $f \in S \to \mathcal{P}(S)$, Cantor's diagonal argument involves the diagonal set $\{x \in S \mid x \notin f(x)\}$. The **grind** tactic takes care of many reasoning steps:

```
S : Type u_1
f : S → Set S
h : Surjective f
⊢ False
```

```
theorem cantor (f : S → Set S) : ¬ Surje
  intro h
  have ⟨x, p⟩ := h (fun x : S => x ∉ f x
  have : x ∈ f x ↔ x ∉ f x := by
    constructor <;>
    simp [Membership.mem] at * <;>
    grind
  grind
```

Verso Blueprint: A Map for Large Formalizations

- Large projects like FLT (50+ contributors) and sphere packing need more than proofs. They need a **map**.
- Tracks definitions, lemmas, and dependencies.
- The blueprint is a Lean document with inline code that is type-checked.
- When AI generates 200,000+ lines of formalization, humans need tools to **visualize and understand** what was built.
- Pure Lean. Built by Emilio Jesús Gallego Arias (Lean FRO).
- Emilio ported: FLT, Carleson, Sphere Packing, Noperthedron.
- **Released this week.**

verso-blueprint

Verso Blueprint: Noperthedron

Rupert Counterexample

Search...

▼ **Table of Contents**

1. Introduction
2. The Noperthedron
3. Bounding Rotations
4. Preliminaries
5. The Global Theorem
6. The Local Theorem
7. Rational Versions
8. Computational Step
9. Main Theorems

Dependency Graph

Blueprint Summary

Blueprint Bibliography

▼ 2. The Noperthedron

2.1. Definition of the Noperthedron

2.2. Refined Rupert's property for the Noperthedron

2.1. Definition of the Noperthedron

Definition 2.1. group used by 0 ✓ LEAN

We define three points $C_1, C_2, C_3 \in \mathbb{Q}^3$.

$$C_1 := \frac{1}{259375205}\begin{pmatrix}152024884\\0\\210152163\end{pmatrix}, \qquad C_2 := \frac{1}{10^{10}}\begin{pmatrix}66327\\61069\\39809\end{pmatrix}$$

$$C_3 := \frac{1}{10^{10}}\begin{pmatrix}8193990033\\5298215096\\1230614493\end{pmatrix}.$$

«def:noperthedron_main»

ASSOCIATED LEAN DECLARATIONS

NopertInline.C1	[complete]
NopertInline.C2	[complete]
NopertInline.C3	[complete]
NopertInline.C1R	[complete]
NopertInline.C2R	[complete]
NopertInline.C3R	[complete]

Code for Definition 2.1.

```
def C1 : Fin 3 → ℚ := (1/259375205) * ![152024884, 0, 210152163]
def C2 : Fin 3 → ℚ := (1/10^10) * ![6632738028, 6106948881, 3980949609]
def C3 : Fin 3 → ℚ := (1/10^10) * ![8193990033, 5298215096, 1230614493]

noncomputable
def C1R : EuclideanSpace ℝ (Fin 3) := WithLp.toLp 2 (fun i => C1 i)

noncomputable
def C2R : EuclideanSpace ℝ (Fin 3) := WithLp.toLp 2 (fun i => C2 i)

noncomputable
def C3R : EuclideanSpace ℝ (Fin 3) := WithLp.toLp 2 (fun i => C3 i)
```

Lemma 2.2. group used by 1 ✓ LEAN

$\|C_1\| = 1$, $\frac{98}{100} < \|C_2\| < \frac{99}{100}$, and $\frac{98}{100} < \|C_3\| < \frac{99}{100}$.

Style blueprint

lean-lang.org

- **lean-lang.org** — main website, written in Verso
- Downloads, documentation, blog, roadmap, team
- **lean-lang.org/fro/roadmap/y3/** — public roadmap
- Next roadmap coming next August
- **lean-lang.org/fro/team/** — the Lean FRO team
- Everything is open source
- **Lean GitHub**

The Lean FRO

Nonprofit. 20 engineers. Open source. Controlled by no single company.

Sebastian Ullrich and I launched it in August 2023.

28 releases and 8,000+ pull requests merged since its launch.

ACM SIGPLAN Award 2025: "*Lean has become the de facto choice for AI-based systems of mathematical reasoning*."

Lean by Example

Types and Pattern Matching

```
inductive Tree (α : Type u) where
  | leaf
  | node (left : Tree α) (value : α) (right : Tree α)
  deriving Inhabited, Repr

def Tree.map (f : α → β) : Tree α → Tree β
  | .leaf => .leaf
  | .node left value right =>
    .node (map f left) (f value) (map f right)

#eval Tree.map (2*·) (.node .leaf 3 (.node .leaf 5 .leaf))
```

```
Tree.node (Tree.leaf) 6 (Tree.node (Tree.leaf) 10 (Tree.leaf))
```

Theorems

Three proofs of the same theorem.

```
theorem Tree.map_id₁ (tree : Tree α) : tree.map id = tree := by
  induction tree with
  | leaf => rfl
  | node l v r ih_l ih_r => simp [map, *]

theorem Tree.map_id₂ (tree : Tree α) : tree.map id = tree := by
  fun_induction Tree.map <;> simp [*]

theorem Tree.map_id₃ (tree : Tree α) : tree.map id = tree := by
  try?
```

Theorems

Three proofs of the same theorem.

```
theorem Tree.map_compose₁ (tree : Tree α) (f : α → β) (g : β → γ)
    : (tree.map f).map g = tree.map (g ∘ f) := by
  induction tree with
  | leaf => rfl
  | node l v r ih_l ih_r => simp [map, *]

theorem Tree.map_compose₂ (tree : Tree α) (f : α → β) (g : β → γ)
    : (tree.map f).map g = tree.map (g ∘ f) := by
  fun_induction Tree.map f tree <;> simp [map, *]

theorem Tree.map_compose₃ (tree : Tree α) (f : α → β) (g : β → γ)
    : (tree.map f).map g = tree.map (g ∘ f) := by
  try?
```

Structures

```
structure Point (α : Type u) where
  x : α
  y : α

inductive Color where
  | red | green | blue

structure ColorPoint (α : Type u) extends Point α where
  c : Color

def p : ColorPoint Nat :=
  { x := 10, y := 20, c := .red }

example : p.x = 10 := rfl
example : p.y = 20 := rfl
example : p.c = .red := rfl

#eval p.x + p.y
```

```
30
```

Typeclasses

```
class Add (α : Type) where
  add : α → α → α

export Add (add)
#check add
```

```
Ex.Add.add {α : Type} [self : Add α] : α → α → α
```

```
instance : Add Nat where
  add := Nat.add

instance : Add Float where
  add := Float.add

instance [Add α] [Add β] : Add (α × β) where
  add := fun (x₁, y₁) (x₂, y₂) => (add x₁ x₂, add y₁ y₂)

def double [Add α] (a : α) : α :=
  Add.add a a

#eval double (2, 3.14, -2.1)
```

```
(4, 6.280000, -4.200000)
```

Typeclasses

```
class Monoid (α : Type u) extends Mul α, One α where
  assoc : ∀ a b c : α, a * (b * c) = (a * b) * c
  one_mul : ∀ a : α, 1 * a = a
  mul_one : ∀ a : α, a * 1 = a

def pow [Monoid α] (a : α) (n : Nat) : α :=
  match n with
  | 0 => 1
  | n+1 => a * pow a n

theorem pow_add [Monoid α] (a : α) (n m : Nat)
  : pow a (n+m) = pow a n * pow a m := by
induction n with
| zero => simp [pow, Monoid.one_mul]
| succ n ih => grind [pow, Monoid.assoc]
```

`Option` and `do` Notation

```
def find? (p : α → Bool) (xs : List α) : Option α := do
  for x in xs do
    if not (p x) then
      continue
    return x
  failure

#eval find? (· > 5) [1, 3, 5, 7, 10]
```

```
some 7
```

Inductive Predicates

Inductive propositions define relations by their introduction rules. This is the key idea behind our big-step semantics.

```
mutual
inductive Even : Nat → Prop
  | zero : Even 0
  | succ : Odd n → Even (n+1)

inductive Odd : Nat → Prop
  | succ : Even n → Odd (n+1)
end

example : Odd 3 := .succ (.succ (.succ .zero))

example : Odd 17 := by repeat constructor

theorem Even.add_two (h : Even n) : Even (n + 2) :=
  .succ (.succ h)

theorem even_or_odd (n : Nat) : Even n ∨ Odd n := by
  induction n with
  | zero => exact .inl .zero
  | succ n ih =>
    cases ih with
    | inl h => exact .inr (.succ h)
    | inr h => exact .inl (.succ h)
```

Metaprogramming: Syntax Extensions

Lean lets you define new syntax and elaborate it into Lean terms. Example: Idiom brackets are an alternative syntax for working with applicative functors.

```
syntax (name := idiom) "⟦" (term:arg)+ "⟧" : term

-- ⟦f a b⟧ ==> f <*> a <*> b
macro_rules
  | `(⟦$f $args*⟧) => do
    let mut out ← `(pure $f)
    for arg in args do
      out ← `($out <*> $arg)
    return out

def addFirstThird [Add α] (xs : List α) : Option α :=
  ⟦Add.add xs[0]? xs[2]?⟧

#eval addFirstThird [10]
```

```
none
```

```
#eval addFirstThird [10,20,30,40]
```

```
some 40
```

This is how we'll build the Mini-Radix DSL syntax.

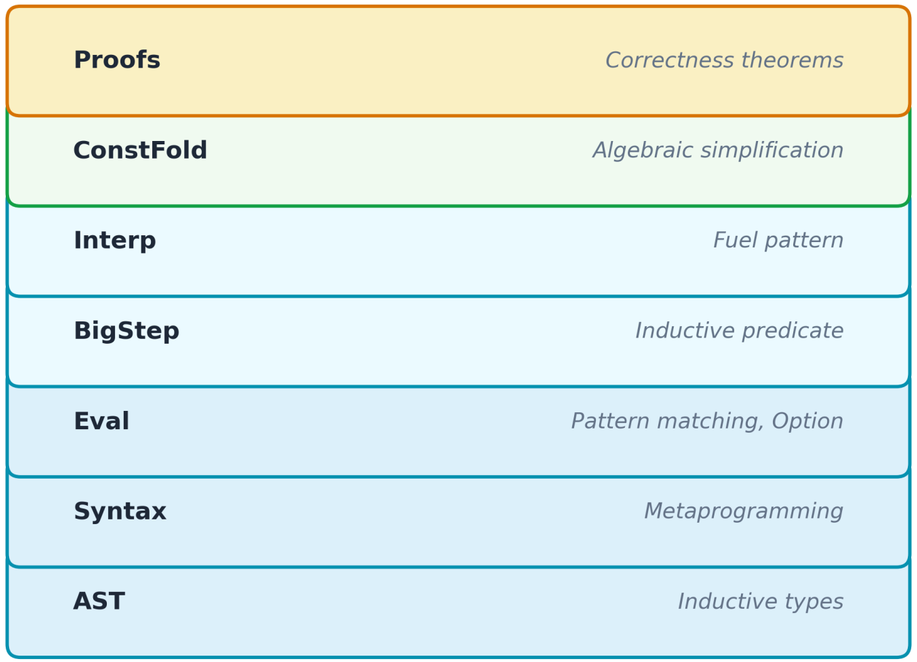
Summary: Lean Features We'll Use

- **Inductive types** — AST: `Expr`, `Stmt`, `Value`
- **Pattern matching** — Evaluator, interpreter
- **`Option` + `do`** — Expression evaluation
- **Syntax macros** — `[RExpr\|...]`, `[RStmt\|...]`
- **Inductive propositions** — Big-step semantics
- **`simp`, `omega`, `cases`** — Correctness proofs

Let's Build Something Real

What We're Building

A simplified version of **Radix**. Each layer teaches a Lean concept:



Layer 1: The AST — What Programs Look Like

Types and Values

Two types: 64-bit unsigned integers and booleans. That's enough to build interesting programs.

```
inductive Ty where
  | uint64
  | bool
  deriving DecidableEq, Repr

inductive Value where
  | uint64 : UInt64 → Value
  | bool : Bool → Value
  deriving DecidableEq, Repr, BEq
```

Operators

Seven binary operators and one unary operator.

```
inductive BinOp where
  | add | sub | mul
  | lt | eq
  | and | or
  deriving DecidableEq, Repr

inductive UnaryOp where
  | not
  deriving DecidableEq, Repr
```

Expressions

Four constructors: literal values, variable lookup, binary operations, unary operations.

```
inductive Expr where
  | lit : Value → Expr
  | var : String → Expr
  | binop : BinOp → Expr → Expr → Expr
  | unop : UnaryOp → Expr → Expr
  deriving Repr, DecidableEq
```

Statements

Five statement forms: skip, assignment, sequencing, if-then-else, while. No heap, no function calls. The `;;` notation lets us write `S₁ ;; S₂`.

```
inductive Stmt where
  | skip
  | assign : String → Expr → Stmt
  | seq : Stmt → Stmt → Stmt
  | ite : Expr → Stmt → Stmt → Stmt
  | while : Expr → Stmt → Stmt
  deriving Repr, DecidableEq

infixr:130 " ;; " => Stmt.seq
```

Layer 2: Concrete Syntax — Making It Readable

Expression Syntax

Reuses Lean's `term` parser for expressions. Each `macro_rules` pattern translates concrete syntax to AST constructors.

```
syntax "`[RExpr|" term "]" : term

macro_rules
  | `(`[RExpr| true])      => `(Expr.lit (.bool true))
  | `(`[RExpr| false])     => `(Expr.lit (.bool false))
  | `(`[RExpr| $n:num])    => `(Expr.lit (.uint64 $n))
  | `(`[RExpr| $x:ident])  => `(Expr.var $(Lean.quote x.getId.toString))
  | `(`[RExpr| ($x)])      => `(`[RExpr| $x])
  | `(`[RExpr| $x + $y])   => `(Expr.binop .add `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x - $y])   => `(Expr.binop .sub `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x * $y])   => `(Expr.binop .mul `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x < $y])   => `(Expr.binop .lt `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x == $y])  => `(Expr.binop .eq `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x && $y])  => `(Expr.binop .and `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| $x || $y])  => `(Expr.binop .or `[RExpr| $x] `[RExpr| $y])
  | `(`[RExpr| !$x])       => `(Expr.unop .not `[RExpr| $x])
```

Statement Syntax

A custom syntax category `rstmt` with its own grammar rules. Semicolons terminate assignments. Braces delimit blocks.

```
syntax ident " := " term ";" : rstmt
syntax "skip" ";" : rstmt
syntax "if" " (" term ")" " {" rstmt* "}" " else" " {" rstmt* "}" : rstmt
syntax "while" " (" term ")" " {" rstmt* "}" : rstmt

syntax "`[RStmt|" rstmt* "]" : term

macro_rules
  | `(`[RStmt| ]) => `(Stmt.skip)
  | `(`[RStmt| skip;]) => `(Stmt.skip)
  | `(`[RStmt| $x:ident := $e:term;]) =>
    `(Stmt.assign $(Lean.quote x.getId.toString) `[RExpr| $e])
  | `(`[RStmt| if ($c) { $ts* } else { $es* }]) =>
    `(Stmt.ite `[RExpr| $c] `[RStmt| $ts*] `[RStmt| $es*])
  | `(`[RStmt| while ($c) { $bs* }]) =>
    `(Stmt.while `[RExpr| $c] `[RStmt| $bs*])
  | `(`[RStmt| $s $ss*]) =>
    `(Stmt.seq `[RStmt| $s] `[RStmt| $ss*])
```

Using the DSL

Same AST, readable syntax. This is exactly how the full Radix DSL works, just with more statement forms.

```
def sumTo := `[RStmt|
  n := 100;
  sum := 0;
  while (0 < n) {
    sum := sum + n;
    n := n - 1;
  }
]
```

Layer 3: Expression Evaluation — Running Expressions

Why Functions as Data Structures?

We could use `HashMap String Value` for the environment. Instead:

```
def Env := String → Option Value
```

- No `HashMap`, no `RBMap`. Just a function.
- `set` returns a new function: `fun y => if y = x then some v else σ y`
- This gives us `@[simp]` lemmas **for free**: `simp` can reason about variable lookup without any data structure internals.
- The proofs essentially write themselves.
- We use efficient data-structures in Radix.

The Environment

```
def Env := String → Option Value

namespace Env

def empty : Env := fun _ => none

def set (σ : Env) (x : String) (v : Value) : Env :=
  fun y => if y = x then some v else σ y

@[simp] theorem set_same (σ : Env) (x : String) (v : Value) :
    (σ.set x v) x = some v := by
  simp [set]

@[simp] theorem set_other (σ : Env) (x y : String) (v : Value) (h : y ≠ x) :
    (σ.set x v) y = σ y := by
  simp [set, h]

end Env
```

Operator Evaluation

Returns `none` on type mismatch (e.g. `true + 3`). Marked `@[simp]` so proofs can reduce these automatically.

```
@[simp] def BinOp.eval : BinOp → Value → Value → Option Value
  | .add, .uint64 a, .uint64 b => some (.uint64 (a + b))
  | .sub, .uint64 a, .uint64 b => some (.uint64 (a - b))
  | .mul, .uint64 a, .uint64 b => some (.uint64 (a * b))
  | .lt,  .uint64 a, .uint64 b => some (.bool (decide (a < b)))
  | .eq,  a,         b         => some (.bool (a == b))
  | .and, .bool a,   .bool b   => some (.bool (a && b))
  | .or,  .bool a,   .bool b   => some (.bool (a || b))
  | _, _, _ => none

@[simp] def UnaryOp.eval : UnaryOp → Value → Option Value
  | .not, .bool b => some (.bool !b)
  | _, _ => none
```

Expression Evaluation

Uses `do` notation for `Option`. If any sub-expression fails, the whole evaluation fails. Expressions are pure: they read `σ` but never modify it.

```
@[simp] def Expr.eval (σ : Env) (e : Expr) : Option Value := do
  match e with
  | .lit v => some v
  | .var x => σ x
  | .binop op l r =>
    let vl ← l.eval σ
    let vr ← r.eval σ
    op.eval vl vr
  | .unop op e =>
    let v ← e.eval σ
    op.eval v

def e₁ : Expr := `[RExpr| 2*a + 1 ]

#eval e₁.eval .empty
```

```
none
```

```
def env : Env := Env.empty.set "a" (.uint64 3)

#eval e₁.eval env
```

```
some (Value.uint64 7)
```

Layer 4: Big-Step Semantics — What Programs Mean

What Are Big-Step Semantics?

A **relation** `BigStep σ s σ'` defined by rules:

"Starting in state `σ`, statement `s` terminates and produces state `σ'`."

- It is a **specification**, not an executable program.
- Each rule describes one statement form.
- We can prove properties **about** the semantics.
- This is the ground truth for correctness proofs.

The BigStep Relation

```
inductive BigStep : Env → Stmt → Env → Prop where
  | skip :
    BigStep σ .skip σ

  | assign (he : e.eval σ = some v) :
    BigStep σ (.assign x e) (σ.set x v)

  | seq (h₁ : BigStep σ₁ s₁ σ₂) (h₂ : BigStep σ₂ s₂ σ₃) :
    BigStep σ₁ (s₁ ;; s₂) σ₃

  | ifTrue (hc : c.eval σ = some (.bool true)) (ht : BigStep σ t σ') :
    BigStep σ (.ite c t f) σ'

  | ifFalse (hc : c.eval σ = some (.bool false)) (hf : BigStep σ f σ') :
    BigStep σ (.ite c t f) σ'

  | whileTrue (hc : c.eval σ₁ = some (.bool true))
      (hb : BigStep σ₁ b σ₂) (hw : BigStep σ₂ (.while c b) σ₃) :
    BigStep σ₁ (.while c b) σ₃

  | whileFalse (hc : c.eval σ = some (.bool false)) :
    BigStep σ (.while c b) σ

notation:60 "⟨" σ ", " s "⟩" " ⇓ " σ':60 => BigStep σ s σ'
```

The BigStep Relation (Example)

```
def σ₁ : Env  := .empty
def s₁ : Stmt := `[RStmt| a := 0; skip; b := a + 1; ]
def σ₂ : Env  := σ₁.set "a" (.uint64 0) |>.set "b" (.uint64 1)

example : BigStep σ₁ s₁ σ₂ := by
  apply BigStep.seq
  apply BigStep.assign
  simp
  rfl
  apply BigStep.seq
  apply BigStep.skip
  apply BigStep.assign
  rfl
```

Why Both a Relation and an Interpreter?

- The **relation** (BigStep) is the specification. It says **what** is correct.
- The **interpreter** is the implementation. It **runs** programs.
- We prove they agree: soundness and completeness.
- This separation is standard in PL: you can change the interpreter without changing the spec.

The relational semantics is the ground truth. Everything else is verified against it.

Progress Check

What we have so far:

- AST: `Expr`, `Stmt`, `Value`, `BinOp`, `UnaryOp`
- Syntax: `[RExpr\|...]`, `[RStmt\|...]`
- Evaluator: `Expr.eval`
- Semantics: `BigStep` (7 rules)

What's left:

- **Interpreter**: make it run (`Stmt.interp`)
- **Constant folding**: optimize safely (`Stmt.constFold`)
- **Proofs**: show everything agrees

Layer 5: The Interpreter — Making It Run

Why the Fuel Pattern?

- BigStep is a **relation**. You can't run it — it is not a program.
- We need an executable version for testing and `#eval`.
- While loops may not terminate. We use the **fuel pattern**: pass a `Nat` that decreases on each step.
- `fuel = 0` means "give up" (return `none`).
- Simple and well-understood. Makes the soundness/completeness proofs straightforward.
- Then we prove the interpreter **agrees** with BigStep.

The Fuel-Based Interpreter

```
def Stmt.interp (fuel : Nat) (s : Stmt) (σ : Env) : Option Env :=
  match fuel with
  | 0 => none
  | fuel + 1 =>
    match s with
    | .skip => some σ
    | .assign x e =>
      match e.eval σ with
      | some v => some (σ.set x v)
      | none => none
    | .seq s₁ s₂ =>
      match s₁.interp fuel σ with
      | some σ' => s₂.interp fuel σ'
      | none => none
  | .ite c t f =>
    match c.eval σ with
    | some (.bool true) => t.interp fuel σ
    | some (.bool false) => f.interp fuel σ
    | _ => none
  | .while c b =>
    match c.eval σ with
    | some (.bool true) =>
      match b.interp fuel σ with
      | some σ' => s.interp fuel σ'
      | none => none
    | some (.bool false) => some σ
    | _ => none
```

```
def Stmt.run (s : Stmt) (fuel : Nat := 1000) : Option Env :=
  s.interp fuel Env.empty
```

Computation Rules

All proved by `rfl`. These are the equations we use in proofs instead of `unfold`.

```
@[simp] theorem Stmt.interp_zero : Stmt.interp 0 s σ = none := rfl

@[simp] theorem Stmt.interp_skip : Stmt.interp (n + 1) .skip σ = some σ := rfl

@[simp] theorem Stmt.interp_assign :
    (Stmt.assign x e).interp (n + 1) σ =
    match e.eval σ with | some v => some (σ.set x v) | none => none := rfl

@[simp] theorem Stmt.interp_seq :
    (s₁ ;; s₂).interp (n + 1) σ =
    match s₁.interp n σ with | some σ' => s₂.interp n σ' | none => none := rfl

@[simp] theorem Stmt.interp_ite :
    (Stmt.ite c t f).interp (n + 1) σ =
    match c.eval σ with
    | some (.bool true) => t.interp n σ
    | some (.bool false) => f.interp n σ
    | _ => none := rfl

@[simp] theorem Stmt.interp_while :
    (Stmt.while c b).interp (n + 1) σ =
    match c.eval σ with
    | some (.bool true) =>
      match b.interp n σ with
      | some σ' => (Stmt.while c b).interp n σ'
      | none => none
```

```
  | some (.bool false) => some σ
  | _ => none := rfl
```

Running Programs

```
def Stmt.runGet (s : Stmt) (x : String) (fuel : Nat := 1000) : Option Value :=
  s.run fuel |>.bind (· x)

def factorial := `[RStmt|
  n := 10;
  result := 1;
  while (0 < n) {
    result := result * n;
    n := n - 1;
  }
]

#eval factorial.runGet "result"
```

```
some (Value.uint64 3628800)
```

```
#eval factorial.runGet "result" (fuel := 0)
```

```
none
```

Layer 6: Constant Folding — Optimizing Safely

What is Constant Folding?

An optimization pass that evaluates constant expressions at compile time:

- `3 + 4` becomes `7`
- `true && e` becomes `e` (if `e` is boolean)
- `e + 0` becomes `e` (if `e` is uint64)
- `if (true) { s₁ } else { s₂ }` becomes `s₁`

The challenge: identity simplifications like `e + 0 → e` require **type information**. We need to know `e` produces a `uint64`.

Quiz: Is `e * 0 → 0` a Valid Optimization?

It looks safe: anything times zero is zero. But consider:

- What if `e.eval σ = none` (e.g., `e` uses an undefined variable)?
- Original: `(e * 0).eval σ = none` (evaluation fails at `e`)
- Optimized: `(0).eval σ = some (.uint64 0)` (succeeds!)

The optimization changes the behavior: it turns a failing program into a succeeding one.

This is why we need `inferTag`: identity rules like `e + 0 → e` are only safe when we **know** `e` will produce a value of the right type.

```
def buggy := `[RStmt|
  n := e * 0;
]

#eval buggy.runGet "n"
```

```
none
```

Type Tags and Inference

```
inductive ValTag where
  | uint64 | bool
  deriving DecidableEq, Repr

@[simp] def Value.tag : Value → ValTag
  | .uint64 _ => .uint64
  | .bool _   => .bool

@[simp] def BinOp.resultTag : BinOp → ValTag → ValTag → Option ValTag
  | .add, .uint64, .uint64
  | .sub, .uint64, .uint64
  | .mul, .uint64, .uint64 => some .uint64
  | .lt, .uint64, .uint64  => some .bool
  | .eq, _, _              => some .bool
  | .and, .bool, .bool
  | .or, .bool, .bool      => some .bool
  | _, _, _                => none

@[simp] def UnaryOp.resultTag : UnaryOp → ValTag → Option ValTag
  | .not, .bool => some .bool
  | _, _        => none
```

Type Tags and Inference

Variables return `none`: we don't track types in the environment. Conservative but sound.

```
def Expr.inferTag : Expr → Option ValTag
  | .lit v        => some v.tag
  | .var _        => none
  | .binop op l r => do
    let tl ← l.inferTag
    let tr ← r.inferTag
    op.resultTag tl tr
  | .unop op e    => do
    let t ← e.inferTag
    op.resultTag t

def c₁ : Expr := `[RExpr| 2*3 - 2 ]
def c₂ : Expr := `[RExpr| true && false ]

#eval c₁.inferTag
```

```
some (ValTag.uint64)
```

```
#eval c₂.inferTag
```

```
some (ValTag.bool)
```

BinOp Simplification

The `if e.inferTag = some .uint64` guard ensures we only simplify when type-safe.

```
def BinOp.simplify : BinOp → Expr → Expr → Expr
    -- Constant folding (both literals)
    | .add, .lit (.uint64 a), .lit (.uint64 b) => .lit (.uint64 (a + b))
    -- ... 5 more literal cases ...
    -- Identity: e + 0 = e (guarded by type tag)
    | .add, e, .lit (.uint64 0) =>
      if e.inferTag == some .uint64 then e
      else .binop .add e (.lit (.uint64 0))
    -- ... more identity cases ...
    | op, a, b => .binop op a b
```

Statement-Level Folding

```
def Expr.constFold : Expr → Expr
  | .lit v => .lit v
  | .var x => .var x
  | .binop op l r => BinOp.simplify op l.constFold r.constFold
  | .unop op e => UnaryOp.simplify op e.constFold

def Stmt.constFold : Stmt → Stmt
  | .skip => .skip
  | .assign x e => .assign x e.constFold
  | .seq s₁ s₂ => .seq s₁.constFold s₂.constFold
  | .ite c t f =>
    match c.constFold with
    | .lit (.bool true) => t.constFold
    | .lit (.bool false) => f.constFold
    | c' => .ite c' t.constFold f.constFold
  | .while c b =>
    match c.constFold with
    | .lit (.bool false) => .skip
    | c' => .while c' b.constFold
```

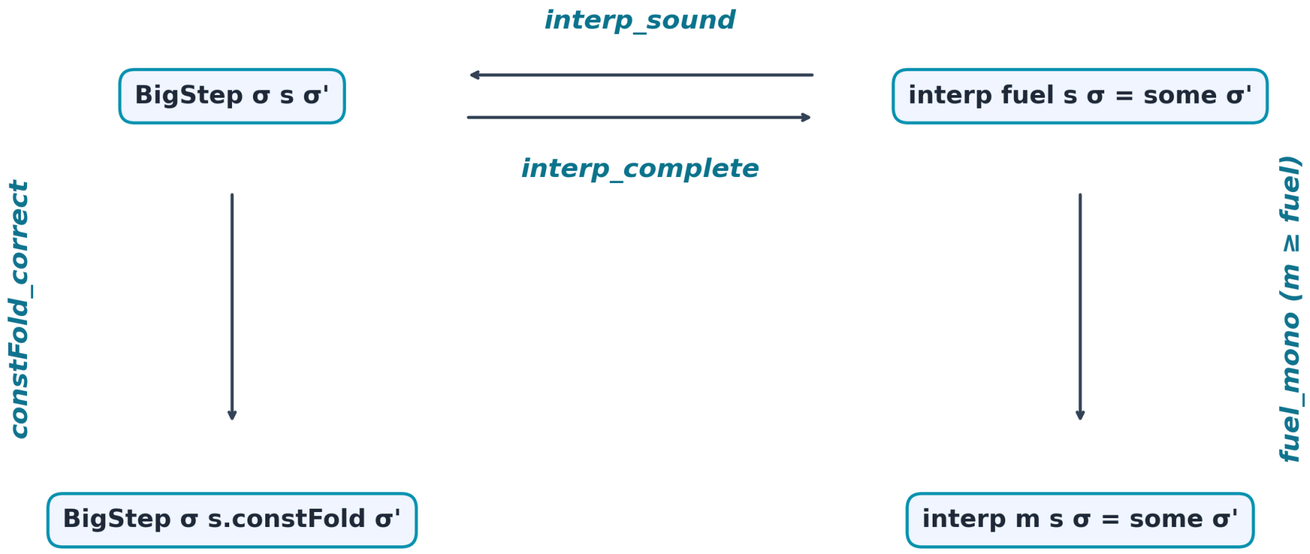
Constant Folding in Action

```
def deadBranch := `[RStmt|
  if (true) {
    x := 3 + 4;
  } else {
    x := 0;
  }
]

-- After constant folding: just `x := 7`
example : deadBranch.constFold = Stmt.assign "x" (.lit (.uint64 7)) := by
  decide
```

Layer 7: Correctness Proofs — Trusting the Code

What We Prove

- `Expr.eval_constFold` : folded expression evaluates the same
- `Stmt.constFold_correct` : folded statement has same BigStep behavior
- `Stmt.interp_sound` : interpreter success implies BigStep
- `Stmt.interp_fuel_mono` : more fuel never breaks a successful run
- `Stmt.interp_complete` : BigStep implies interpreter success (with enough fuel)


interp_sound
BigStep σ s σ'
interp fuel s σ = some σ'
interp_complete
constFold_correct
fuel_mono (m ≥ fuel)
BigStep σ s.constFold σ'
interp m s σ = some σ'

Expression-Level Correctness

Structural induction. Once we have `BinOp.simplify_correct` as a `@[simp]` lemma, the proof is straightforward.

```
theorem expr_eval_constFold (e : Expr) (σ : Env) :
    e.constFold.eval σ = e.eval σ := by
  induction e with
  | lit v => simp [Expr.constFold, Expr.eval]
  | var x => simp [Expr.constFold, Expr.eval]
  | binop op l r ihl ihr =>
    simp only [Expr.constFold, BinOp.simplify_correct, Expr.eval, ihl, ihr]
  | unop op e ih =>
    simp only [Expr.constFold, UnaryOp.simplify_correct, Expr.eval, ih]
```

Statement-Level Correctness

Induction on `BigStep`. Rewrite condition using `Expr.eval_constFold`, then case-split on what `c.constFold` reduces to.

```
theorem constFold_correct (h : BigStep σ s σ') : BigStep σ s.constFold σ' := by
  induction h with
  | skip => exact BigStep.skip
  | assign he =>
    simp only [Stmt.constFold]
    exact BigStep.assign (by rw [expr_eval_constFold]; exact he)
  | seq _ _ ih₁ ih₂ =>
    simp only [Stmt.constFold]; exact BigStep.seq ih₁ ih₂
  | ifTrue hc _ ih =>
    simp only [Stmt.constFold]
    have hc' := hc; rw [← expr_eval_constFold] at hc'
    split
    · exact ih
    · next heq => rw [heq, Expr.eval] at hc'; cases hc'
    · exact BigStep.ifTrue hc' ih
  | ifFalse hc _ ih =>
    simp only [Stmt.constFold]
    have hc' := hc; rw [← expr_eval_constFold] at hc'
    split
    · next heq => rw [heq, Expr.eval] at hc'; cases hc'
    · exact ih
    · exact BigStep.ifFalse hc' ih

-- ..
```

```
| whileTrue hc _ _ ihb ihw =>
  simp only [Stmt.constFold] at ihw ⊢
  have hc' := hc; rw [← expr_eval_constFold] at hc'
  split at *
  · next heq => rw [heq, Expr.eval] at hc'; cases hc'
  · exact BigStep.whileTrue hc' ihb ihw
| whileFalse hc =>
  simp only [Stmt.constFold]
  have hc' := hc; rw [← expr_eval_constFold] at hc'
  split
  · exact BigStep.skip
  · exact BigStep.whileFalse hc'
```

Interpreter Soundness

Full proof **here**

```
theorem interp_sound {fuel : Nat} {s : Stmt} {σ σ' : Env}
    (h : s.interp fuel σ = some σ') :
    BigStep σ s σ' := by
  apply Stmt.interp_sound h
```

Interpreter Completeness

The `seq` case is the most interesting: take the max of both fuels, then use fuel monotonicity to lift both results.

Full proofs **here**

```
theorem interp_fuel_mono {n m : Nat} (h : n ≤ m)
    {s : Stmt} {σ σ' : Env}
    (hok : s.interp n σ = some σ') :
    s.interp m σ = some σ' := by
  apply Stmt.interp_fuel_mono h hok

theorem interp_complete
  {σ : Env} {s : Stmt} {σ' : Env}
  (h : BigStep σ s σ') :
  ∃ fuel : Nat, s.interp fuel σ = some σ' := by
apply Stmt.interp_complete h
```

Proof Techniques Summary

- **Induction** on expressions for evaluation lemmas
- **Induction** on `BigStep` for statement correctness
- **Induction** on fuel for interpreter proofs
- `simp only [...]` with `@[simp]` computation rules
- `cases` on `Option` values to handle success/failure
- `omega` for natural number arithmetic
- `decide` for decidable propositions (testing)

What We Left Out

Mini-Radix is deliberately minimal. We excluded:

- **Heap allocation**: `malloc`, `free`, arrays — adds memory safety proofs
- **Function calls**: call stack, scoping — adds determinism complexity
- **Linear ownership**: tracking who owns what — prevents use-after-free
- **More optimizations**: copy propagation, dead code elimination, inlining
- **String type**

The full Radix handles all of them.

What the Full Radix Adds

- **Types**: uint64, bool → + string
- **Heap**: No → malloc, free, arrays
- **Functions**: No → function calls, stack
- **Ownership**: No → linear ownership typing
- **Optimizations**: 1 (const fold) → 5 verified passes
- **BigStep rules**: 7 → 16
- **Lines**: ~950 → ~7,400
- **Theorems**: 5 → 52
- `sorry`: 0 → 0

Where to Go Next

- **This repo**: all code from today
- **Full Radix**: **github.com/leodemoura/RadixExperiment**
- **Install Lean**: **lean-lang.org**
- **Learn**: **Functional Programming in Lean**, **Theorem Proving in Lean 4**, **Lean Reference Manual**
- **Community**: **Lean Zulip**
- **Lean FRO**: **lean-lang.org/fro/**
- **Why Lean?**

What We Built Today

- A complete verified DSL in ~950 lines of Lean
- Custom syntax, expression evaluation, big-step semantics
- A fuel-based interpreter, proved sound and complete
- An optimization pass, proved correct
- Zero `sorry`. Every claim is machine-checked.

AI built the full Radix in a weekend. You can build Mini-Radix in an afternoon.

Fork the repo. Add function calls. Prove it correct.

Thank You

Leo de Moura

lean-lang.org

leanprover.zulipchat.com

X: @leanprover

LinkedIn: Lean FRO

Mastodon: @leanprover@functional.cafe

#leanlang #leanprover

leodemoura.github.io

ETAPS 2026